

Wall-X LeRobot Media VisPruner 实验报告

Wall-X / VisPruner 实验整理

2026-05-27

目录

摘要	1
实验设置	1
方法说明	2
总体结果	2
图片样本分析	2
视频样本分析	3
结论与建议	4

摘要

本报告整理了 Wall-X 模型在 LIBERO 多媒体样本上的 VisPruner 计时实验。实验比较了不裁剪的基线版本与启用 VisPruner 后的版本，重点观察 token 数量、总推理耗时和主要阶段耗时的变化。

在图片样本中，平均视觉 token 从 81.000 降到 41.000，减少 49.38%；平均总耗时从 334.586 ms 降到 332.378 ms，约减少 0.66%。这说明当前裁剪确实减少了图片 token，但总耗时收益较小，主要原因是整体时间由后续 ODE/Transformer 计算占主导。

在视频样本中，token 数量保持 324.000 不变；平均总耗时从 423.138 ms 降到 411.939 ms，约减少 2.65%。需要注意，当前实现的硬裁剪只接入图片 token，视频路径本身没有执行 token 裁剪，因此视频结果更适合作为路径计时参考，而不能解读为视频 token 裁剪效果。

实验设置

项目	取值
数据集	libero_all
模型路径	/root/autodl-tmp/wall_x/pretrained/wall-oss-fast

项目	取值
图片样本数	40
视频样本数	20
每个视频抽取帧数	8
预热次数 / 计时迭代	1 / 3
保留比例 keep_ratio	0.5
运行设备	cuda

方法说明

实验分别运行两个版本：基线版本保留全部视觉 token；裁剪版本在图片路径中根据得分选择部分 token。每个样本先经过预热，再进行多次计时迭代，报告中的时间为平均值，单位均为毫秒。为了便于阅读，正的耗时差值表示裁剪版本更慢，负的耗时差值表示裁剪版本更快。

总体结果

token 数量变化如下：

样本类型	样本数	裁剪前 token	裁剪后 token	token 差值	token 变化
图片样本	40	81.000	41.000	-40.000	-49.38%
视频样本	20	324.000	324.000	0.000	0.00%

端到端总耗时变化如下：

样本类型	基线总耗时 (ms)	裁剪总耗时 (ms)	耗时差 (ms)	耗时变化
图片样本	334.586	332.378	-2.209	-0.66%
视频样本	423.138	411.939	-11.199	-2.65%

从总体结果看，图片路径的 token 数量明显下降，但端到端总耗时只小幅下降；视频路径没有发生 token 裁剪。

图片样本分析

40 个图片样本中，有 27 个样本在启用裁剪后更快，13 个样本更慢。单样本耗时差的平均值为 -2.209 ms，中位数为 -1.655 ms。这说明图片路径的收益存在波动，但总体方向是轻微加速。

图片路径中耗时变化最大的阶段如下：

阶段	基线 (ms)	裁剪 (ms)	差值 (ms)	变化
vision_image_encode	26.279	0.000	-26.279	-100.00%
vision_image_encode_score	0.000	24.522	+24.522	0.00%
total_time	334.586	332.378	-2.209	-0.66%
ode_integration	276.006	275.024	-0.982	-0.36%
ode_transformer_total	269.724	268.754	-0.970	-0.36%
vispruner_total	0.000	0.804	+0.804	0.00%
position_encoding	0.620	0.108	-0.512	-82.58%
pruning_position_ids_prepare	0.000	0.504	+0.504	0.00%
position_ids_rope	0.491	0.000	-0.491	-100.00%
embed_processing	26.768	26.363	-0.405	-1.51%
image_path_total	26.370	25.993	-0.376	-1.43%
vision_image_forward	26.395	26.020	-0.375	-1.42%

视频样本分析

20 个视频样本中，有 20 个样本在裁剪配置下更快，0 个样本更慢。单样本耗时差的平均值为 -11.199 ms，中位数为 -9.294 ms。由于视频 token 未被裁剪，这部分结果主要反映同一模型视频路径在两次配置下的运行差异。

视频路径中耗时变化最大的阶段如下：

阶段	基线 (ms)	裁剪配置 (ms)	差值 (ms)	变化
vision_video_forward	107.405	91.231	-16.174	-15.06%
embed_processing	107.880	91.750	-16.131	-14.95%
total_time	423.138	411.939	-11.199	-2.65%
ode_integration	282.757	286.635	+3.878	+1.37%
ode_transformer_total	276.246	279.280	+3.034	+1.10%
prefetch_forward	29.898	30.767	+0.869	+2.91%
prefill_transformer	29.696	30.540	+0.844	+2.84%
external_prepare_batch_ms	19.597	19.141	-0.456	-2.33%
ode_action_embed_total	2.723	3.074	+0.352	+12.91%
ode_action_head_total	0.975	1.180	+0.205	+21.05%
cache_preprocessing	1.267	1.339	+0.073	+5.73%
position_encoding	0.736	0.806	+0.070	+9.51%

结论与建议

本次实验表明，VisPruner 对图片 token 的削减效果明确：在 keep_ratio 为 0.5 时，图片 token 数量约减少一半。不过，端到端总耗时只下降约 0.66%，说明当前瓶颈并不完全在图片 token 编码阶段，而更多集中在 ODE 积分和 Transformer 相关计算中。后续如果希望获得更明显的速度收益，可以重点考虑三点：第一，将裁剪收益传递到更多后续计算阶段；第二，继续优化裁剪本身带来的额外开销；第三，为视频路径补充真正的 token 裁剪逻辑，再单独评估视频效果。